

Nate Riggs
CS-470
Southern New Hampshire University
08/17/2026
YouTube Video Link: <https://youtu.be/QJ4P5CVmkb0>

CS-470 Final Reflection

Experiences and Strengths

CS-470 has directly shaped where I want to take my career. I'm currently working in a data-focused role, and this course has confirmed that the data field, whether that ends up being data engineering, data analysis, or data science, is where I want to stay. The work in this course, particularly around structuring DynamoDB tables, writing Lambda functions to process and query that data, and thinking carefully about how data moves between a frontend, an API layer, and a database, lines up closely with the kind of problem-solving I've been doing in my current role and want to keep doing going forward.

One of the biggest technical jumps for me in this course was learning Docker and containerization. Before CS-470, I understood the concept of “it works on my machine” problems in the abstract, but actually building out Dockerfiles and a multi-container setup with Docker Compose made it concrete. I now have hands-on experience packaging an application so it runs the same way regardless of environment, which is a skill that matters well beyond web development, it's just as relevant when you're standing up reproducible data pipelines or environments for analysis work.

As a developer, my biggest strength is being methodical. I care less about getting something to work once and more about making sure it works the same way every time, reliably, from a consistent starting point. That mindset showed up throughout this course: when I hit CORS configuration issues or Lambda permission errors, I didn't move on until I understood

exactly why something failed and confirmed the fix held up under repeated testing, not just a lucky first pass. That habit of insisting on repeatability rather than settling for “it worked once” is something I bring to every project, and it's especially valuable in data work, where a script or pipeline that works today but silently breaks tomorrow can cause real damage down the line.

Coming out of CS-470, I feel prepared to take on roles that blend backend development with data infrastructure: data engineer roles that involve building and maintaining pipelines, cloud-based data analyst roles that require comfort with AWS services like Lambda and DynamoDB, or hybrid software engineering roles where reliable, well-tested backend systems are the priority.

Planning for Growth

Looking at the Questions and Answers application I built and migrated to a serverless architecture, there's a clear path for how I'd plan its future growth using the cloud concepts covered in this course.

Scale and error handling. The current architecture already scales automatically at the compute layer: each Lambda function spins up additional concurrent executions as request volume increases, without any manual provisioning on my part. As the application grows, I'd extend this by adding dead-letter queues to catch failed Lambda invocations, so errors get captured and retried or logged instead of silently disappearing. I'd also add CloudWatch alarms tied to error rates and Lambda throttling, so I'd know immediately if a spike in traffic started hitting concurrency limits, rather than finding out from a user complaint. On the database side, DynamoDB's on-demand capacity mode, which is what I used, already absorbs traffic spikes

without me needing to predict throughput ahead of time, but at higher scale I'd evaluate switching to provisioned capacity with auto-scaling for more predictable cost control.

Cost prediction. Serverless costs are usage-based: I pay per Lambda invocation, per GB-second of compute time, and per API Gateway request. That makes cost highly variable in the short term, since a sudden traffic spike directly increases the bill, but it's also proportionate to actual value delivered. Containers, by contrast, are more cost-predictable because you're paying for a fixed amount of allocated compute whether it's fully utilized or not, a flat monthly cost regardless of traffic. So containers are more cost predictable in the sense of a stable, forecastable bill, while serverless is more cost efficient in that you never pay for idle capacity, but it requires more careful monitoring to avoid surprise costs during unexpected usage spikes.

Pros and cons for expansion. Sticking with a fully serverless approach as the app grows has real advantages: no server management, automatic scaling, and costs that track usage rather than a flat baseline. The tradeoffs are cold-start latency on infrequently used Lambda functions, harder local debugging compared to a traditional server, and a ceiling on execution time (15 minutes per Lambda invocation) that makes it a poor fit if the application ever needed to run long-running background jobs. Moving some components to containers, for example, running a container in ECS or Fargate for a component with more predictable, constant traffic, would trade away some of that automatic elasticity in exchange for more predictable performance and cost for that specific piece, while keeping the bursty, unpredictable parts of the app on Lambda where pay-for-use makes more sense.

Elasticity and pay-for-service in decision making. These two principles are really what decide the split between serverless and containers as the app grows. Elasticity matters most for

parts of the application with unpredictable or spiky traffic, since Lambda and DynamoDB's on-demand mode absorb that variability without any manual intervention. Pay-for-service matters most when I'm trying to control cost during a growth phase where usage is still uncertain, since it means the infrastructure bill grows in step with actual adoption rather than requiring me to guess capacity months in advance and either overpay for unused headroom or underprovision and hurt the user experience. As the application matures and traffic patterns become more predictable, I'd revisit that balance and consider shifting steady, high-volume components to containers or provisioned capacity, where a flat, predictable cost becomes more advantageous than the flexibility of pure pay-for-use.